

Question description

Given two rooted trees, your task is to find out if they are *isomorphic*, i.e., it is possible to draw them so that they look the same.

Constraints

- $1 \leq t \leq 1000$
- $2 \leq n \leq 10^5$
- the sum of all values of n is at most 10^5

Input

The first input line has an integer t : the number of tests. Then, there are t tests described as follows:
The first line has an integer n : the number of nodes in both trees. The nodes are numbered $1, 2, \dots, n$, and node 1 is the root.
Then, there are $n-1$ lines describing the edges of the first tree, and finally $n-1$ lines describing the edges of the second tree.

Output

For each test, print "YES", if the trees are isomorphic, and "NO" otherwise.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
2
3
1 2
2 3
1 2
1 3
3
1 2
2 3
1 3
3 2
```

Test Case 2

INPUT (STDIN)

```
2
3
1 2
2 3
1 3
3 3
3
1 2
2 3
1 3
3 3
```

EXPECTED OUTPUT

NO
YES

EXPECTED OUTPUT

NO
NO

▼ Mandatory Test Cases

Test Case 1

KEYWORD

```
while(t--)
```

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

2

Test Case 2

TOKEN COUNT

480

Test Case 3

NLOC

65

```
1
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <stdint.h>
5
6 typedef struct {
7     int *head;
8     int *to;
9     int *nxt;
10    int edge_cnt;
11} Graph;
12
13void init_graph(Graph *g, int n) {
14    g->head = (int*)malloc((n + 1) * sizeof(int));
15    g->to = (int*)malloc(2 * n * sizeof(int));
16    g->nxt = (int*)malloc(2 * n * sizeof(int));
17    for (int i = 0; i <= n; i++) g->head[i] = -1;
18    g->edge_cnt = 0;
19}
20
21void add_edge(Graph *g, int u, int v) {
22    g->to[g->edge_cnt] = v;
23    g->nxt[g->edge_cnt] = g->head[u];
24    g->head[u] = g->edge_cnt++;
25}
```

```

26
27void free_graph(Graph *g) {
28    free(g->head);
29    free(g->to);
30    free(g->nxt);
31}
32
33int cmp_u64(const void *a, const void *b) {
34    uint64_t x = *(const uint64_t*)a;
35    uint64_t y = *(const uint64_t*)b;
36    if (x < y) return -1;
37    if (x > y) return 1;
38    return 0;
39}
40
41static inline uint64_t splitmix64(uint64_t x) {
42    x += 0x9e3779b97f4a7c15ULL;
43    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
44    x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;

```

```

45     return x ^ (x >> 31);
46}
47
48uint64_t dfs(int u, int p, Graph *g) {
49     uint64_t child_hashes[64];
50     uint64_t *arr = child_hashes;
51     int cap = 64;
52     int cnt = 0;
53
54     for (int e = g->head[u]; e != -1; e = g->nxt[e]) {
55         int v = g->to[e];
56         if (v == p) continue;
57         if (cnt >= cap) {
58             cap *= 2;
59
60             for (int i = 0; i < cnt; i++) new_arr[i] = arr[i];
61             if (arr != child_hashes) free(arr);
62             arr = new_arr;

```

```

63     }
64     arr[cnt++] = dfs(v, u, g);
65 }
66
67 qsort(arr, cnt, sizeof(uint64_t), cmp_u64);
68
69 uint64_t h = 14695981039346656037ULL;
70 for (int i = 0; i < cnt; i++) {
71     h ^= splitmix64(arr[i]);
72     h *= 1099511628211ULL;
73 }
74 h ^= (uint64_t)cnt;
75
76 if (arr != child_hashes) free(arr);
77 return h;
78}
79
80int main() {

```

```
81  int t;
82  if (scanf("%d", &t) != 1) return 0;
83  while (t--) {
84      int n;
85      if (scanf("%d", &n) != 1) break;
86      Graph g1, g2;
87      init_graph(&g1, n);
88      init_graph(&g2, n);
89
90      for (int i = 0; i < n - 1; i++) {
91          int u, v;
92          if (scanf("%d %d", &u, &v) == 2) {
93              add_edge(&g1, u, v);
94              add_edge(&g1, v, u);
95          }
96      }
97      for (int i = 0; i < n - 1; i++) {
98          int u, v;
```

```
99         if (scanf("%d %d", &u, &v) == 2) {
100             add_edge(&g2, u, v);
101             add_edge(&g2, v, u);
102         }
103     }
104
105     uint64_t h1 = dfs(1, 0, &g1);
106     uint64_t h2 = dfs(1, 0, &g2);
107
108     if (h1 == h2) printf("YES\n");
109     else printf("NO\n");
110
111     free_graph(&g1);
112     free_graph(&g2);
113 }
114 return 0;
115 }
116
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

2

3

1 2

2 3

1 2

1 3

3

1 2

2 3

1 3

3 2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree21.exe

NO

YES

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

2

3

1 2

2 3

1 3

3 3

3

1 2

2 3

1 3

3 3

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree21.exe

NO

NO

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```